

Network Latency in Distributed LLM Inference

PROGRESS UPDATE: APRIL 15, 2026

Kelsey Penners

Multi-GPU Experiments Set-Up

Model: Llama 3.1-8B (baseline)
Servers: shyfec (NVLink, SHM, Network Socket)
TP configurations: 2, 4 on **shy-fec** (A100s)
Workload:

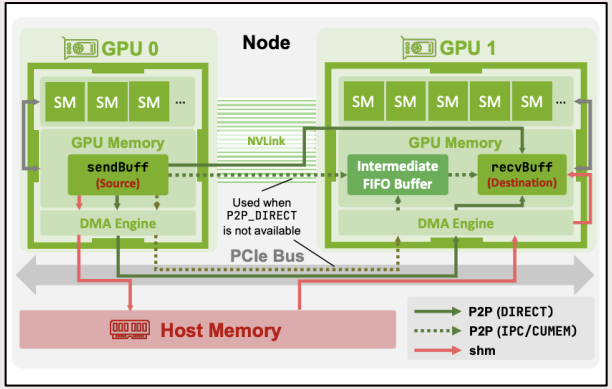
Concurrency sweep continued with fixed ISL/OSL, continuing with random prompts

Latency injection: TBD

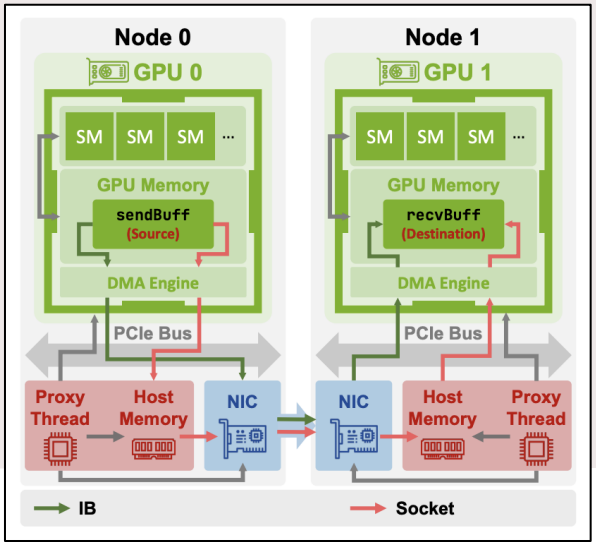
Funnotch (V100) experiments blocked for now.

NCCL Transport

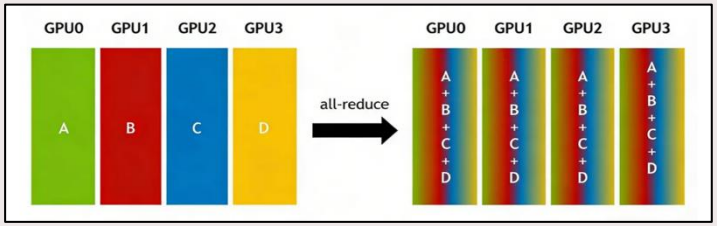
- NCCL prioritizes lowest latency, highest bandwidth paths available (often P2P)
- Shared memory (SHM):
  - Data routed via system memory
  - GPU → PCIe → Host Mem → PCIe → GPU
- Inter-node communication:
  - Managed by CPU proxy thread
  - Uses TCP sockets or Infiniband
- Socket-based communication:
  - Multiple memory copies
  - GPU → PCIe → Host Mem → TCP stack → loopback → TCP stack → Host Mem → PCIe → GPU.



source: https://arxiv.org/pdf/2507.04786v1



Communication in TP



Source: "In-Depth Understanding of AI Distributed Training Communication Primitives"

Tensor Parallelism:

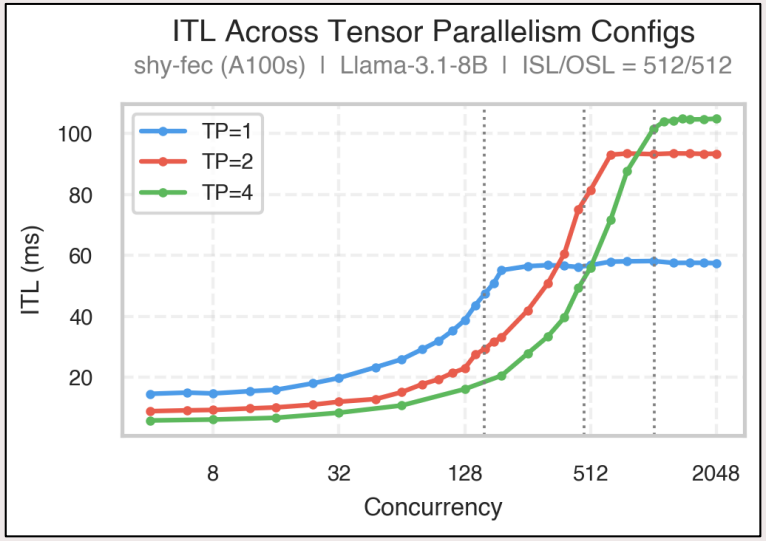
- Each transformer layer is split across GPUs
- Per layer communication: 2 x AllReduce ops per layer
  - One after attention, one after MLP

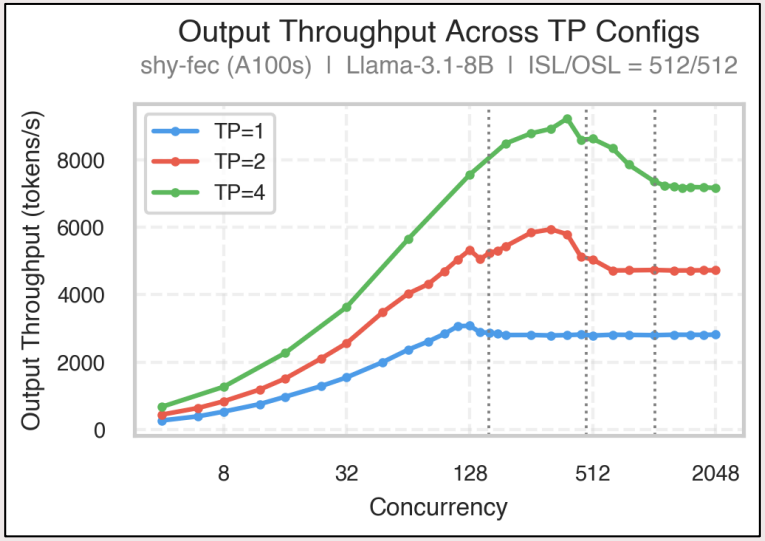
For Llama 3.1 8B (32 layers) in BF16:

- TP frequency per forward pass = 2 * 32 = 64 AllReduce ops
- *Decode:* each step (per token) has 64 sequential AllReduce calls
  - Total calls = 64 * output sequence length
- ITL overhead from communication ≈ 64 * AllReduce latency

- Different parallelism strategies (PP, MoE) have different communication patterns

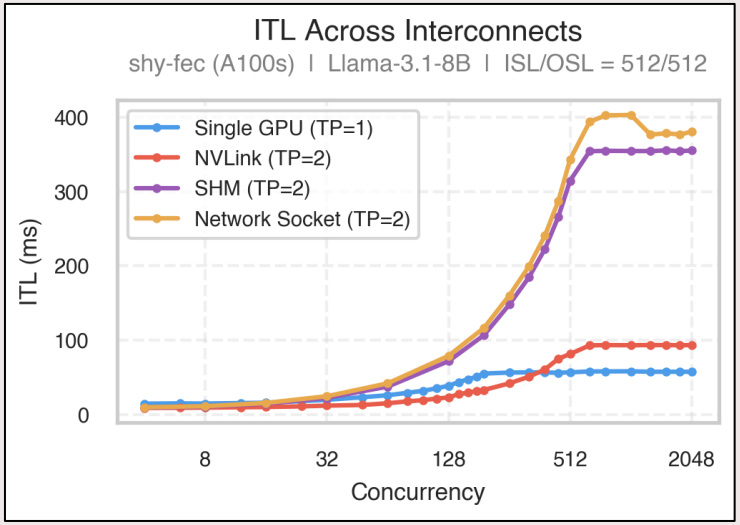
Multi-GPU: Across TP Configs

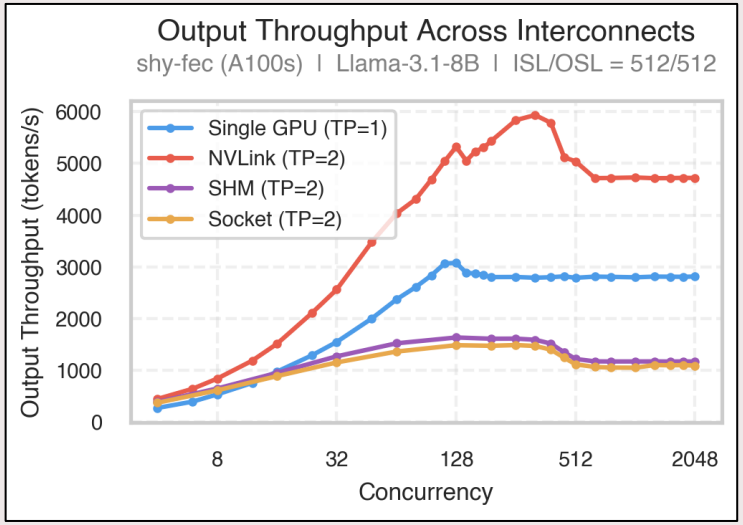




TP scaling shifts saturation point and increases output throughput.
Approx. KV cache saturation points for TP=1,2,4 : 150, 450, 1000

Multi-GPU: Across Interconnects





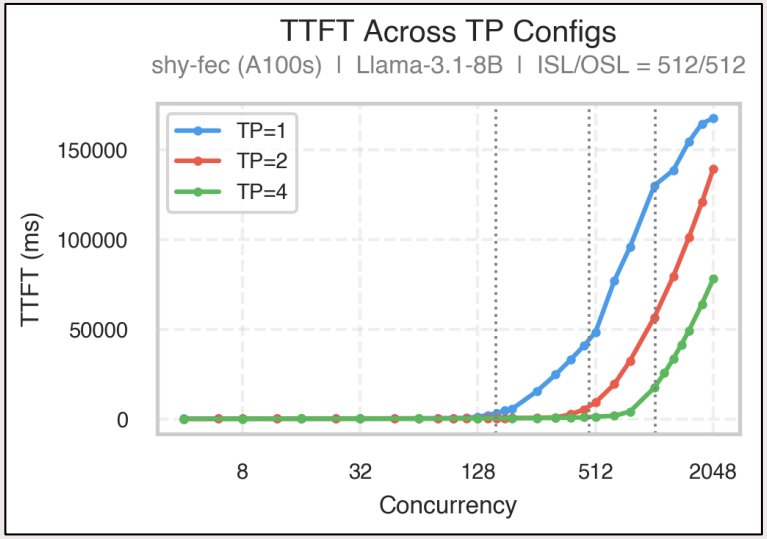
SHM & socket transport nearly identical, both require copying of data to memory.

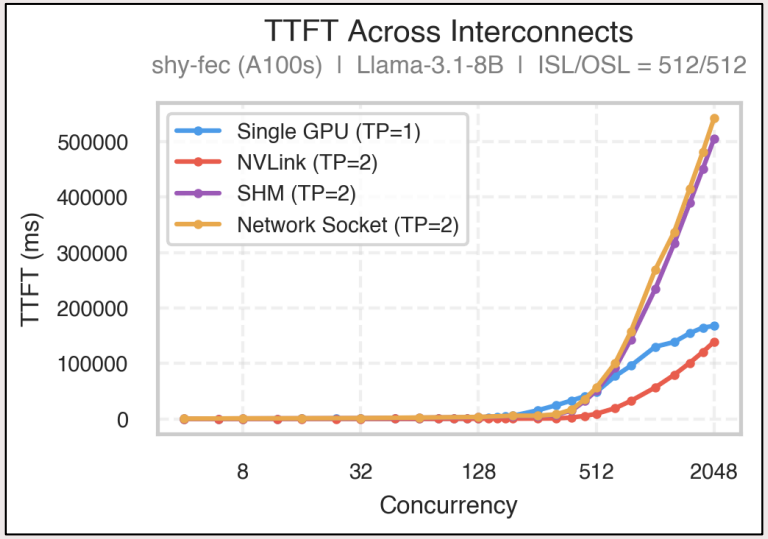
Discussion Points

- Latency injection approach?
  - `tc netem` requires communication to go through network stack
  - Wrapping NCCL calls in vLLM source code
- Model scope?
  - Compare against an MoE (OLMoE or Mixtral) here? Or a larger model?
- Thesis direction?
  - Current framing: empirical analysis of LLM behavior in response to KV cache saturation, interconnects, distribution methods
- Citations?

Thank you!

Multi-GPU TTFT Behavior





After saturation point, TTFT begins to continuously climb, resulting in TTFTs unrealistic for real-word deployment

Model of Communication Overhead

$$\mathrm{ITL}_{\mathrm{TP=k}} = \mathrm{ITL}_{\mathrm{compute}} + 64 * \mathrm{L}_{\mathrm{AllReduce}}(k)$$

$$\mathrm{ITL}_{\mathrm{compute}} \approx \mathrm{ITL}_{\mathrm{TP=1}}$$

$$\Delta\mathrm{ITL} = \mathrm{ITL}_{\mathrm{TP=k}} - \mathrm{ITL}_{\mathrm{compute}}$$

$$\mathrm{L}_{\mathrm{observedAllReduce}} = \Delta\mathrm{ITL} \,/\, 64$$

Taking low concurrency,TP=1 as approximately pure compute ITL, we can estimate AllReduce latency. Very oversimplified, as AllReduce latency varies due to message size, and ITL also depends on sequence length.